

Embedded Systems Project Report

3×3 LED Grid Navigation

Using STM32F412ZX Microcontroller

Group Members:

Name	Roll Number
Gattu Charitha	B23EE1021
Mallam Vishnu Priya	B23EE1040
Adepu Srikar	B23EE1001
Bondala Shyam Charan	B23EE1012

Subject: Embedded Systems

Date: April 18, 2026

Contents

1	Introduction	2
2	Objectives	2
3	Hardware Description	2
3.1	Microcontroller	2
3.2	3×3 LED Grid Layout	2
3.3	Hardware Setup	3
3.4	Button-to-Direction Mapping	5
4	Grid Coordinate Model	6
5	Software Design	6
5.1	System Initialization (<code>main</code>)	6
5.2	LED Update Function	7
5.3	EXTI and SYSCFG Configuration	7
5.4	Interrupt Service Routines	8
5.4.1	Cardinal Directions	8
5.4.2	Diagonal Directions	8
5.5	Boundary Conditions	9
6	System Flow	9
7	Results and Observations	10
8	Conclusion	10
9	Full Source Code	11

1. Introduction

This report documents the design and implementation of a **3×3 LED Grid Navigation System** on the STM32F412ZX ARM Cortex-M4 microcontroller. The system uses nine LEDs arranged in a 3×3 matrix to represent a virtual grid cursor. Eight push-buttons provide directional control — four cardinal directions (Up, Down, Left, Right) and four diagonal directions (Up-Left, Up-Right, Down-Left, Down-Right) — allowing the user to move a single illuminated LED across all nine grid positions.

The entire firmware is implemented at the bare-metal register level, with no use of any Hardware Abstraction Layer (HAL). Key embedded concepts covered include GPIO configuration, External Interrupt (EXTI) routing via SYSCFG, NVIC interrupt management, and state-driven LED output using a 2D coordinate model.

2. Objectives

- Model a 3×3 grid using two integer state variables x (row) and y (column).
- Map nine GPIO output pins across GPIOB and GPIOE to the nine grid positions.
- Configure eight GPIO input pins with internal pull-up resistors as directional buttons.
- Route all eight button signals to appropriate EXTI lines via SYSCFG and handle them using four NVIC interrupt vectors.
- Implement bounded navigation logic that prevents the cursor from moving outside the 3×3 grid.
- Demonstrate all eight movement directions (4 cardinal + 4 diagonal) in real time on hardware.

3. Hardware Description

3.1 Microcontroller

The **STM32F412ZX** is based on the ARM Cortex-M4 core with FPU, running at up to 100 MHz. It offers multiple GPIO ports with flexible alternate function and interrupt capabilities, making it ideal for multi-pin real-time input/output applications.

3.2 3×3 LED Grid Layout

The nine LEDs are arranged conceptually as a 3×3 matrix indexed by row $x \in \{0, 1, 2\}$ and column $y \in \{0, 1, 2\}$. Table 1 shows the mapping between grid coordinates and physical GPIO pins.

Table 1: LED Grid Pin Mapping

Row (x)	Column (y)	GPIO Pin	Grid Position
0	0	PB5	Top-Left
0	1	PB9	Top-Centre
0	2	PB3	Top-Right
1	0	PB7	Middle-Left
1	1	PE1	Centre
1	2	PB2	Middle-Right
2	0	PB0	Bottom-Left
2	1	PE2	Bottom-Centre
2	2	PB1	Bottom-Right

3.3 Hardware Setup

Figure 1 shows the actual hardware implementation on a breadboard, with the STM32F412ZX Nucleo board connected to the 3×3 LED grid and the eight directional push-buttons. The button labels visible in the image correspond to: **U** (Up), **D** (Down), **L** (Left), **R** (Right), **U-L** (Up-Left), **U-R** (Up-Right), **D-R** (Down-Right), and **D-L** (Down-Left).

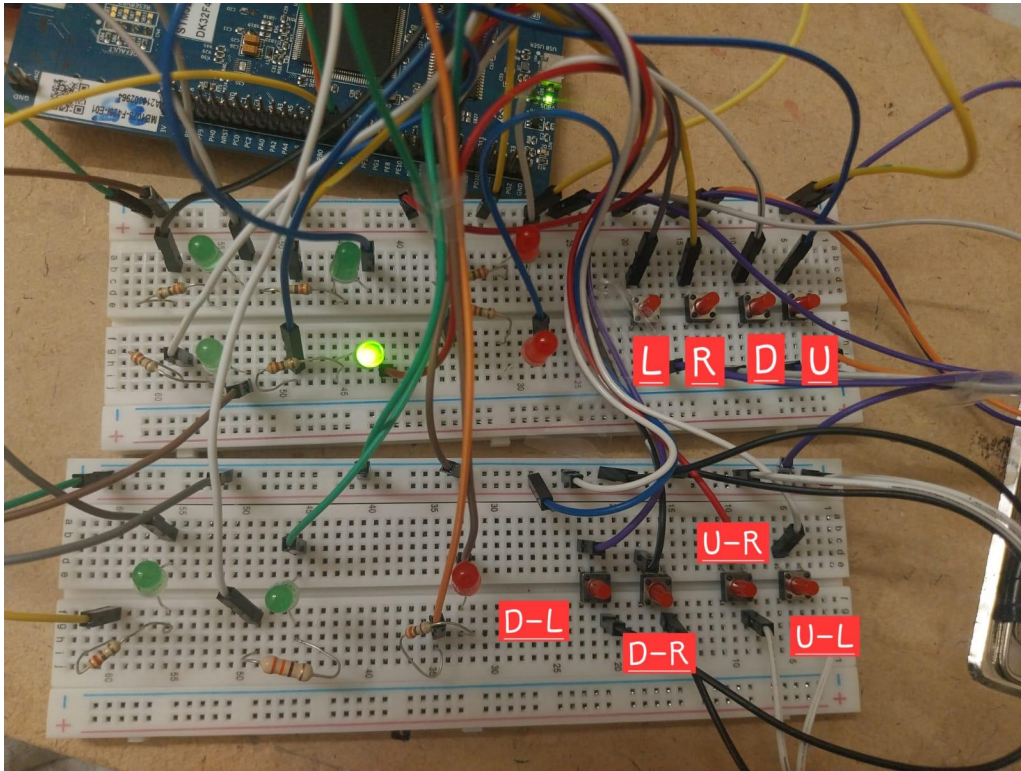


Figure 1: Hardware setup showing the STM32F412ZX board, 3×3 LED grid, and directional push-buttons on breadboard.

Figures 2 and 3 show the system in operation, with the cursor active at two different grid positions demonstrating the green and red LEDs illuminated respectively.

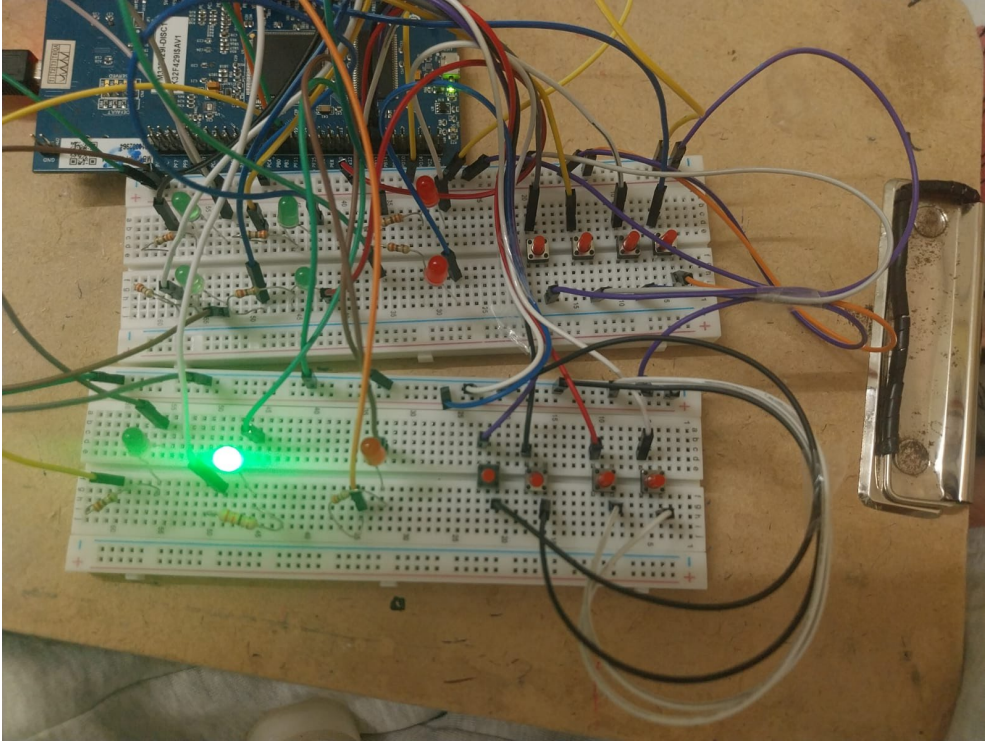


Figure 2: System in operation — green LED illuminated, cursor positioned at a bottom-row grid cell.

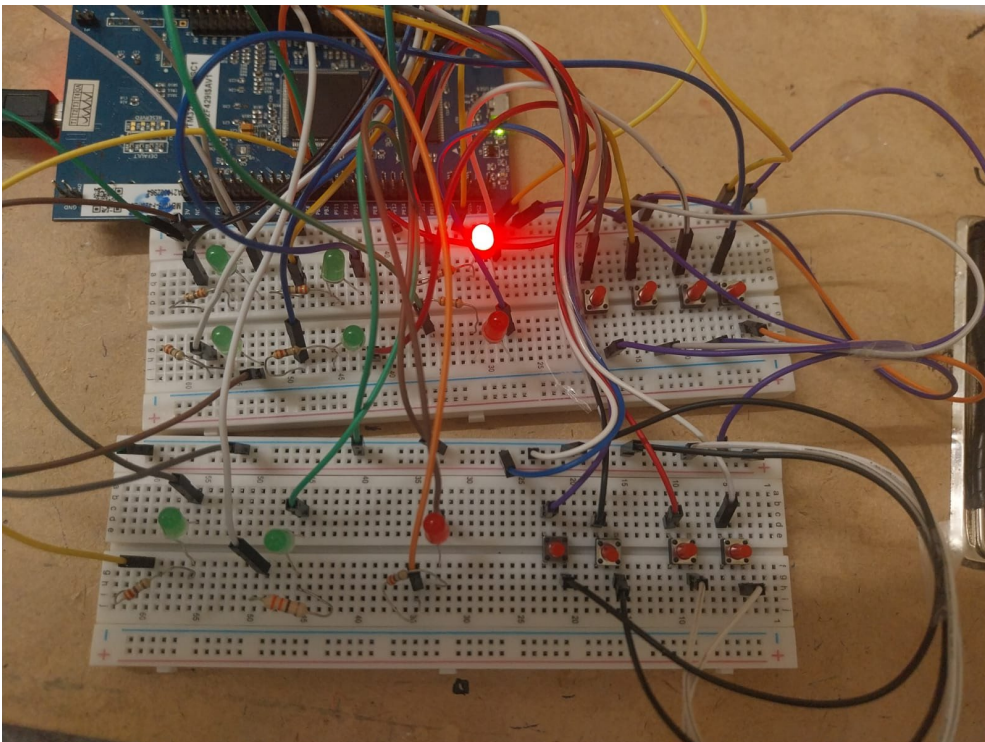


Figure 3: System in operation — red LED illuminated, cursor positioned at a middle-row grid cell.

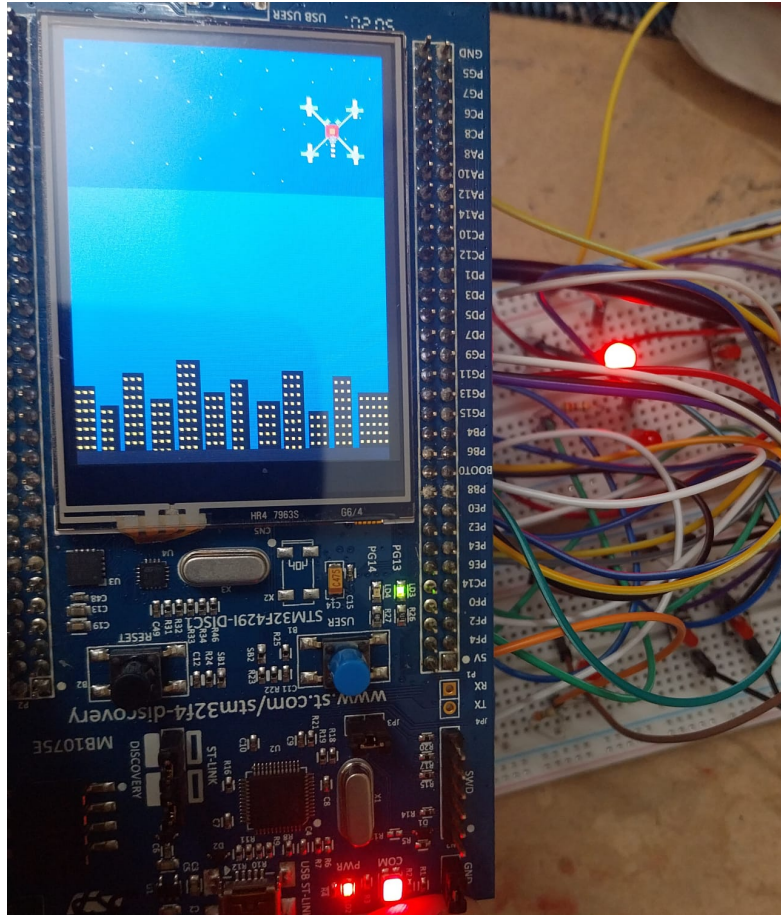


Figure 4: STM32F429I-Discovery board with onboard LCD display active alongside the breadboard LED grid, showing the complete hardware assembly.

3.4 Button-to-Direction Mapping

Eight buttons are used for navigation. Each button is connected to a GPIO pin configured as input with internal pull-up, and generates a falling-edge interrupt when pressed. Table 2 summarises the mapping.

Table 2: Button Pin and Direction Mapping

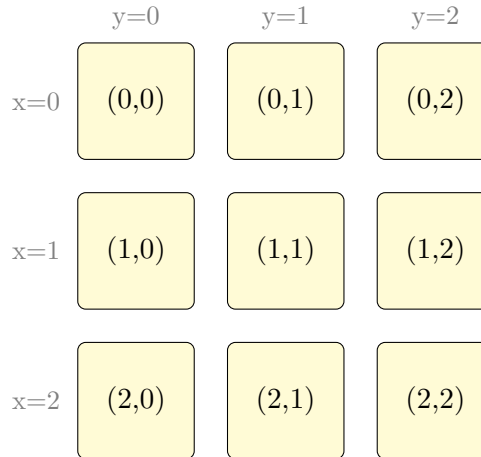
GPIO Pin	EXTI Line	NVIC Vector	Direction
PB6	EXTI6	EXTI9_5_IRQn	Up
PB8	EXTI8	EXTI9_5_IRQn	Right
PB4	EXTI4	EXTI4_IRQn	Down
PE0	EXTI0	EXTI0_IRQn	Left
PB10	EXTI10	EXTI15_10_IRQn	Diagonal Up-Left
PB11	EXTI11	EXTI15_10_IRQn	Diagonal Up-Right
PB12	EXTI12	EXTI15_10_IRQn	Diagonal Down-Right
PB13	EXTI13	EXTI15_10_IRQn	Diagonal Down-Left

4. Grid Coordinate Model

The cursor position on the 3×3 grid is represented by two global integer variables:

- `int x` — the current **row** (0 = top, 2 = bottom)
- `int y` — the current **column** (0 = left, 2 = right)

Movement in any direction modifies `x`, `y`, or both. Boundary checks ensure that neither variable goes below 0 or above 2. The following diagram illustrates the coordinate system:



5. Software Design

5.1 System Initialization (`main`)

The `main()` function performs all peripheral setup before enabling interrupts and entering an infinite idle loop. The initialization sequence is:

1. Disable global interrupts (`__disable_irq()`).
2. Enable clocks for GPIOB, GPIOE, and SYSCFG via RCC registers.
3. Configure nine GPIO pins (PB0–PB3, PB5, PB7, PB9, PE1, PE2) as push-pull outputs for LEDs.
4. Configure eight GPIO pins (PB4, PB6, PB8, PB10–PB13, PE0) as inputs with internal pull-up resistors.
5. Clear and configure SYSCFG EXTICR registers to route GPIO pins to EXTI lines.
6. Set EXTI IMR (interrupt mask) and FTSR (falling edge trigger) for all eight lines.
7. Enable four NVIC interrupt vectors.
8. Re-enable global interrupts and call `update_led()` to display the initial cursor position.
9. Enter `while(1)` idle loop — all activity is interrupt-driven.

5.2 LED Update Function

`update_led()` is the central display routine. It first clears all nine LED output pins atomically using bitmask operations on the ODR register, then sets exactly one pin corresponding to the current (x, y) state:

```

1 void update_led() {
2     /* Clear all LED pins */
3     GPIOB->ODR &= ~( (1<<0) | (1<<1) | (1<<2) | (1<<3) |
4                     (1<<5) | (1<<7) | (1<<9));
5     GPIOE->ODR &= ~( (1<<1) | (1<<2));
6
7     /* Light the LED at current (x, y) position */
8     if(x==0 && y==0) GPIOB->ODR |= (1<<5); // PB5: Top-Left
9     if(x==0 && y==1) GPIOB->ODR |= (1<<9); // PB9: Top-Centre
10    if(x==0 && y==2) GPIOB->ODR |= (1<<3); // PB3: Top-Right
11    if(x==1 && y==0) GPIOB->ODR |= (1<<7); // PB7: Middle-Left
12    if(x==1 && y==1) GPIOE->ODR |= (1<<1); // PE1: Centre
13    if(x==1 && y==2) GPIOB->ODR |= (1<<2); // PB2: Middle-Right
14    if(x==2 && y==0) GPIOB->ODR |= (1<<0); // PB0: Bottom-Left
15    if(x==2 && y==1) GPIOE->ODR |= (1<<2); // PE2: Bottom-Centre
16    if(x==2 && y==2) GPIOB->ODR |= (1<<1); // PB1: Bottom-Right
17 }

```

Listing 1: LED Update Function

5.3 EXTI and SYSCFG Configuration

All SYSCFG EXTICR registers are first cleared to zero to avoid stale mappings from previous configurations. Each button GPIO pin is then routed to its EXTI line:

```

1 SYSCFG->EXTICR[0] = 0; SYSCFG->EXTICR[1] = 0;
2 SYSCFG->EXTICR[2] = 0; SYSCFG->EXTICR[3] = 0;
3
4 SYSCFG->EXTICR[0] |= (0x4 << 0); // PE0 -> EXTI0
5 SYSCFG->EXTICR[1] |= (0x1 << 0); // PB4 -> EXTI4
6 SYSCFG->EXTICR[1] |= (0x1 << 8); // PB6 -> EXTI6
7 SYSCFG->EXTICR[2] |= (0x1 << 0); // PB8 -> EXTI8
8 SYSCFG->EXTICR[2] |= (0x1 << 8); // PB10 -> EXTI10
9 SYSCFG->EXTICR[2] |= (0x1 << 12); // PB11 -> EXTI11
10 SYSCFG->EXTICR[3] |= (0x1 << 0); // PB12 -> EXTI12
11 SYSCFG->EXTICR[3] |= (0x1 << 4); // PB13 -> EXTI13

```

Listing 2: SYSCFG EXTI Routing

Falling-edge triggers are enabled for all eight EXTI lines, and the interrupt mask register is unmasked accordingly:

```

1 EXTI->IMR |= (1<<0) | (1<<4) | (1<<6) | (1<<8) |
2             (1<<10) | (1<<11) | (1<<12) | (1<<13);
3 EXTI->FTSR |= (1<<0) | (1<<4) | (1<<6) | (1<<8) |
4             (1<<10) | (1<<11) | (1<<12) | (1<<13);

```

Listing 3: EXTI Mask and Trigger Configuration

5.4 Interrupt Service Routines

5.4.1 Cardinal Directions

The four cardinal direction ISRs each check the EXTI Pending Register (PR), clear the flag by writing 1 to it, apply a boundary-checked update to *x* or *y*, and call `update_led()`:

```

1  /* UP: PB6 -> EXTI6 (decrements row x) */
2  void EXTI9_5_IRQHandler(void) {
3      if (EXTI->PR & (1<<6)) {
4          EXTI->PR = (1<<6);
5          if(x > 0) x--;
6          update_led();
7      }
8      if (EXTI->PR & (1<<8)) { // RIGHT: PB8 -> EXTI8
9          EXTI->PR = (1<<8);
10         if(y < 2) y++;
11         update_led();
12     }
13 }

14
15 /* DOWN: PB4 -> EXTI4 (increments row x) */
16 void EXTI4_IRQHandler(void) {
17     if (EXTI->PR & (1<<4)) {
18         EXTI->PR = (1<<4);
19         if(x < 2) x++;
20         update_led();
21     }
22 }

23
24 /* LEFT: PE0 -> EXTI0 (decrements column y) */
25 void EXTI0_IRQHandler(void) {
26     if (EXTI->PR & (1<<0)) {
27         EXTI->PR = (1<<0);
28         if(y > 0) y--;
29         update_led();
30     }
31 }

```

Listing 4: Cardinal Direction ISRs

5.4.2 Diagonal Directions

The shared `EXTI15_10_IRQHandler` handles all four diagonal movements. Each diagonal simultaneously modifies both *x* and *y* with appropriate boundary checks:

```

1  void EXTI15_10_IRQHandler(void) {
2      if (EXTI->PR & (1<<10)) { // Up-Left: x--, y--
3          EXTI->PR = (1<<10);
4          if(x>0 && y>0){ x--; y--; }
5          update_led();
6      }
7      if (EXTI->PR & (1<<11)) { // Up-Right: x--, y++

```

```

8      EXTI->PR = (1<<11);
9      if(x>0 && y<2){ x--; y++; }
10     update_led();
11 }
12 if (EXTI->PR & (1<<12)) {    // Down-Right: x++, y++
13     EXTI->PR = (1<<12);
14     if(x<2 && y<2){ x++; y++; }
15     update_led();
16 }
17 if (EXTI->PR & (1<<13)) {    // Down-Left: x++, y--
18     EXTI->PR = (1<<13);
19     if(x<2 && y>0){ x++; y--; }
20     update_led();
21 }
22 }

```

Listing 5: Diagonal Direction ISR

5.5 Boundary Conditions

The navigation logic enforces strict boundary checking before any state update. Table 3 summarises the conditions under which each direction is allowed.

Table 3: Movement Boundary Conditions

Direction	State Change	Condition
Up	x--	x > 0
Down	x++	x < 2
Left	y--	y > 0
Right	y++	y < 2
Up-Left	x--, y--	x > 0 && y > 0
Up-Right	x--, y++	x > 0 && y < 2
Down-Right	x++, y++	x < 2 && y < 2
Down-Left	x++, y--	x < 2 && y > 0

6. System Flow

1. On power-up, all GPIO clocks, pin modes, EXTI routing, and NVIC vectors are configured.
2. `update_led()` is called once to illuminate the LED at the initial position ($x = 0$, $y = 0$) — the top-left corner.
3. The processor enters the `while(1)` idle loop. No polling occurs; the system is entirely interrupt-driven.
4. A button press generates a falling edge on the corresponding GPIO pin, triggering the mapped EXTI line.
5. The appropriate ISR fires, clears the pending bit, updates `x` and/or `y` (if within bounds), and calls `update_led()`.

6. `update_led()` clears all LEDs and sets exactly the one LED corresponding to the new `(x, y)` state.
7. The system returns to idle, ready for the next button press.

7. Results and Observations

- The cursor navigates correctly across all nine grid positions in all eight directions.
- Boundary conditions are correctly enforced — pressing a directional button at an edge has no effect on the cursor position.
- The entirely interrupt-driven architecture means the processor spends virtually all of its time in the idle loop, with zero CPU cycles wasted on polling.
- Clearing all SYSCFG EXTICR registers to zero before configuration prevents ghost mappings that could cause spurious interrupts.
- The diagonal ISR (`EXTI15_10`) correctly uses compound boundary conditions (e.g., `x > 0 && y > 0`) to prevent partial diagonal movement when only one axis is at its boundary.
- A potential limitation is the absence of hardware debouncing; rapid repeated button presses may cause multiple increments. A software debounce timer could be added in a future revision.
- The initial cursor position on startup is always `(0, 0)` (top-left LED), as `x` and `y` are initialised to zero globally.

8. Conclusion

This project successfully implements an 8-directional cursor navigation system on a 3×3 LED grid using the STM32F412ZX microcontroller programmed at bare-metal register level. By combining GPIO output control, EXTI interrupt routing, NVIC management, and a simple 2D state model, the system provides responsive and bounded directional movement across all nine grid positions. The design demonstrates key embedded systems principles including interrupt-driven architecture, peripheral register manipulation, and state-driven I/O — all without relying on any software abstraction layer.

9. Full Source Code

```

1  #include "stm32f412zx.h"
2
3  int x = 0;
4  int y = 0;
5
6  void update_led() {
7      GPIOB->ODR &= ~((1<<0)|(1<<1)|(1<<2)|(1<<3)|
8                      (1<<5)|(1<<7)|(1<<9));
9      GPIOE->ODR &= ~((1<<1)|(1<<2));
10
11     if(x==0 && y==0) GPIOB->ODR |= (1<<5);
12     if(x==0 && y==1) GPIOB->ODR |= (1<<9);
13     if(x==0 && y==2) GPIOB->ODR |= (1<<3);
14     if(x==1 && y==0) GPIOB->ODR |= (1<<7);
15     if(x==1 && y==1) GPIOE->ODR |= (1<<1);
16     if(x==1 && y==2) GPIOB->ODR |= (1<<2);
17     if(x==2 && y==0) GPIOB->ODR |= (1<<0);
18     if(x==2 && y==1) GPIOE->ODR |= (1<<2);
19     if(x==2 && y==2) GPIOB->ODR |= (1<<1);
20 }
21
22 int main() {
23     __disable_irq();
24
25     RCC->AHB1ENR |= (1<<1);    // GPIOB
26     RCC->AHB1ENR |= (1<<4);    // GPIOE
27     RCC->APB2ENR |= (1<<14);   // SYSCFG
28
29     /* LED outputs: PB0,1,2,3,5,7,9 */
30     GPIOB->MODER &= ~(
31         (3<<(0*2))|(3<<(1*2))|(3<<(2*2))|(3<<(3*2))|
32         (3<<(5*2))|(3<<(7*2))|(3<<(9*2))
33     );
34     GPIOB->MODER |= (
35         (1<<(0*2))|(1<<(1*2))|(1<<(2*2))|(1<<(3*2))|
36         (1<<(5*2))|(1<<(7*2))|(1<<(9*2))
37     );
38     GPIOE->MODER &= ~((3<<(1*2))|(3<<(2*2)));
39     GPIOE->MODER |= ((1<<(1*2))|(1<<(2*2)));
40
41     /* Button inputs with pull-up */
42     GPIOB->MODER &= ~((3<<(6*2))|(3<<(4*2))|(3<<(8*2))|
43                     (3<<(10*2))|(3<<(11*2))|(3<<(12*2))|(3<<(13*2)));
44     GPIOB->PUPDR |= ((1<<(6*2))|(1<<(4*2))|(1<<(8*2))|
45                     (1<<(10*2))|(1<<(11*2))|(1<<(12*2))|(1<<(13*2)));
46     GPIOE->MODER &= ~(3<<(0*2));
47     GPIOE->PUPDR |= (1<<(0*2));
48
49     /* SYSCFG EXTI routing */
50     SYSCFG->EXTICR[0] = 0;

```

```

51     SYSCFG->EXTICR[1] = 0;
52     SYSCFG->EXTICR[2] = 0;
53     SYSCFG->EXTICR[3] = 0;
54     SYSCFG->EXTICR[0] |= (0x4<<0);    // PE0 -> EXTI0
55     SYSCFG->EXTICR[1] |= (0x1<<0);    // PB4 -> EXTI4
56     SYSCFG->EXTICR[1] |= (0x1<<8);    // PB6 -> EXTI6
57     SYSCFG->EXTICR[2] |= (0x1<<0);    // PB8 -> EXTI8
58     SYSCFG->EXTICR[2] |= (0x1<<8);    // PB10 -> EXTI10
59     SYSCFG->EXTICR[2] |= (0x1<<12);   // PB11 -> EXTI11
60     SYSCFG->EXTICR[3] |= (0x1<<0);    // PB12 -> EXTI12
61     SYSCFG->EXTICR[3] |= (0x1<<4);    // PB13 -> EXTI13
62
63     EXTI->IMR |= (1<<6)|(1<<4)|(1<<8)|(1<<0)|
64                 (1<<10)|(1<<11)|(1<<12)|(1<<13);
65     EXTI->FTSR |= (1<<6)|(1<<4)|(1<<8)|(1<<0)|
66                 (1<<10)|(1<<11)|(1<<12)|(1<<13);
67
68     NVIC_EnableIRQ(EXTI9_5_IRQn);
69     NVIC_EnableIRQ(EXTI4_IRQn);
70     NVIC_EnableIRQ(EXTIO_IRQn);
71     NVIC_EnableIRQ(EXTI15_10_IRQn);
72
73     __enable_irq();
74     update_led();
75     while(1);
76 }
77
78 void EXTI9_5_IRQHandler(void) {
79     if (EXTI->PR & (1<<6)) {
80         EXTI->PR = (1<<6);
81         if(x > 0) x--;
82         update_led();
83     }
84     if (EXTI->PR & (1<<8)) {
85         EXTI->PR = (1<<8);
86         if(y < 2) y++;
87         update_led();
88     }
89 }
90
91 void EXTI4_IRQHandler(void) {
92     if (EXTI->PR & (1<<4)) {
93         EXTI->PR = (1<<4);
94         if(x < 2) x++;
95         update_led();
96     }
97 }
98
99 void EXTIO_IRQHandler(void) {
100     if (EXTI->PR & (1<<0)) {
101         EXTI->PR = (1<<0);

```

```
102         if(y > 0) y--;  
103         update_led();  
104     }  
105 }  
106  
107 void EXTI15_10_IRQHandler(void) {  
108     if (EXTI->PR & (1<<10)) {  
109         EXTI->PR = (1<<10);  
110         if(x>0 && y>0){ x--; y--; }  
111         update_led();  
112     }  
113     if (EXTI->PR & (1<<11)) {  
114         EXTI->PR = (1<<11);  
115         if(x>0 && y<2){ x--; y++; }  
116         update_led();  
117     }  
118     if (EXTI->PR & (1<<12)) {  
119         EXTI->PR = (1<<12);  
120         if(x<2 && y<2){ x++; y++; }  
121         update_led();  
122     }  
123     if (EXTI->PR & (1<<13)) {  
124         EXTI->PR = (1<<13);  
125         if(x<2 && y>0){ x++; y--; }  
126         update_led();  
127     }  
128 }
```

Listing 6: Complete Firmware — main.c